



Received: 23-03-2025
Accepted: 03-05-2025

International Journal of Advanced Multidisciplinary Research and Studies

ISSN: 2583-049X

Confidential Computing in Front-End: Enhancing Data Security with Secure Enclaves and Homomorphic Encryption

Yuliia Horbenko

Master's degree, Senior Front-End Developer, Smartbarrel, National Transport University, 1 Mykhaila Omelianovycha-Pavlenka Str., Kyiv, 01010, Ukraine

Corresponding Author: **Yuliia Horbenko**

Abstract

This thesis investigates how homomorphic encryption and, more especially, Secure Enclaves might improve data security in front-end web systems using Confidential Computing methods. Conventional encryption techniques are insufficient in maintaining confidentiality during data processing when client-side technologies progressively control sensitive data. This paper evaluates in real-world front-end applications the utility, efficiency, and scalability of Secure Enclaves (e.g., Intel SGX and ARM TrustZone) and numerous Homomorphic Encryption frameworks (including

BFV and CKKS). Analyzing real-world events, including secure form entries and in-browser document management, helps the study assess performance indicators, including latency, CPU use, and integration complexity. Studies show that Secure Enclaves provide better performance and simpler integration; homomorphic encryption improves privacy protections even if it results in higher computing costs. This work ends with analyzing the trade-offs related to different approaches and suggesting suitable use cases for every scenario, thereby improving the safety of data while in use.

Keywords: Confidential Computing, Front-End Security, Secure Enclaves, Homomorphic Encryption, Client-Side Security, Trusted Execution Environment

Introduction

Synopsis of Confidential Computing and Its Applications to Front-End Development

From a simple user interface to a sophisticated computer platform capable of running sensitive activities including financial transactions, health data entry, and identity verification, the fast development of front-end technology has changed the web browser. While data-at-rest protection and HTTPS offer some degree of security, they create a notable void: the protection of data in use. This term refers to the period during which data is actively processed on a user's device or within a web browser, including actions such as validation, transformation, or analytics. Despite the presence of modern encryption protocols, user information may still be vulnerable during this stage due to threats such as malicious browser extensions, compromised JavaScript libraries, or other attack vectors (Sabt & Achemlal, 2015) ^[27]. Confidential computing introduces an architecture specifically designed to protect data throughout this processing period, thereby ensuring its confidentiality even while it is being used. This protection is primarily achieved through Trusted Execution Environments (TEEs) or through advanced cryptographic techniques that isolate or conceal data from unauthorized access during computation (Hunt *et al.*, 2018) ^[17]. The primary objective of this approach is to create a secure enclave where data can be processed regardless of the security posture of the surrounding system. Although confidential computing is gaining widespread adoption in cloud and backend systems, such as secure data processing in untrusted cloud environments, its implementation in front-end development remains in the early stages of exploration. Contemporary front-end applications, particularly those leveraging WebAssembly, Progressive Web Apps (PWAs), and edge computing technologies, are increasingly performing complex logic on the client side. These developments offer both opportunities and challenges. On one hand, they enable fast and interactive services without requiring continuous server communication. On the other hand, they introduce potential risks by exposing sensitive information in potentially insecure browser environments.

Overview of Secure Enclaves and Homomorphic Encryption (HE)

In the field of Confidential Computing, which has to do with the front-end systems, Secure Enclaves and Homomorphic Encryption (HE) are the most interesting and well researched technologies. Because both offer a creative approach to the way data is been protected while in use. Yet, there are variations as to their performances, complexity of integration, and levels of security.

Secure Enclaves

Secure enclaves (Trusted Execution Environments or TEE) are at the core of confidential computing. They are sets of security-related instruction codes built into new CPUs. They protect data in use, because the enclave is decrypted on the fly only within the CPU, and exclusively for code and data running within the enclave itself. In a secure enclave, applications run in an environment isolated from their host, such that memory is allocated in isolation from every other application running on the machine, including the Operating system. These enclaves protect applications, codes, and data while on premises, across the network, and in the cloud. These enclaves secure important operations, including privileged software such as operating systems and hypervisors, from direct exposure to the system (Sabt &

Achemlal, 2015). Even in scenarios where the foundational system may be compromised, these solutions generally aim to maintain the integrity and confidentiality of code and data while in use. Intel Software Guard Extensions (SGX) is one example of this technology, as it enables developers to execute specified code and control data within secure CPU "enclaves." SGX ensures that data within the enclave remains impervious to external access and that any unauthorized attempts to modify the enclave are swiftly detected (Costan & Devadas, 2016) ^[10].

Similarly, ARM TrustZone offers a dual-execution architecture that includes a "secure world" and a "normal world." This structure enables the execution of sensitive operations, such as cryptographic key management and digital rights enforcement, within a protected environment (ARM, 2023). Trusted Execution Environments (TEEs) can be integrated with WebAssembly (Wasm) in front-end applications to perform secure computations close to the user while maintaining isolation. However, several limitations constrain the practical deployment of this technology in web browsers. These include hardware compatibility issues, the lack of native TEE APIs, and inconsistencies across platforms, all of which pose significant integration challenges (Fig 1).

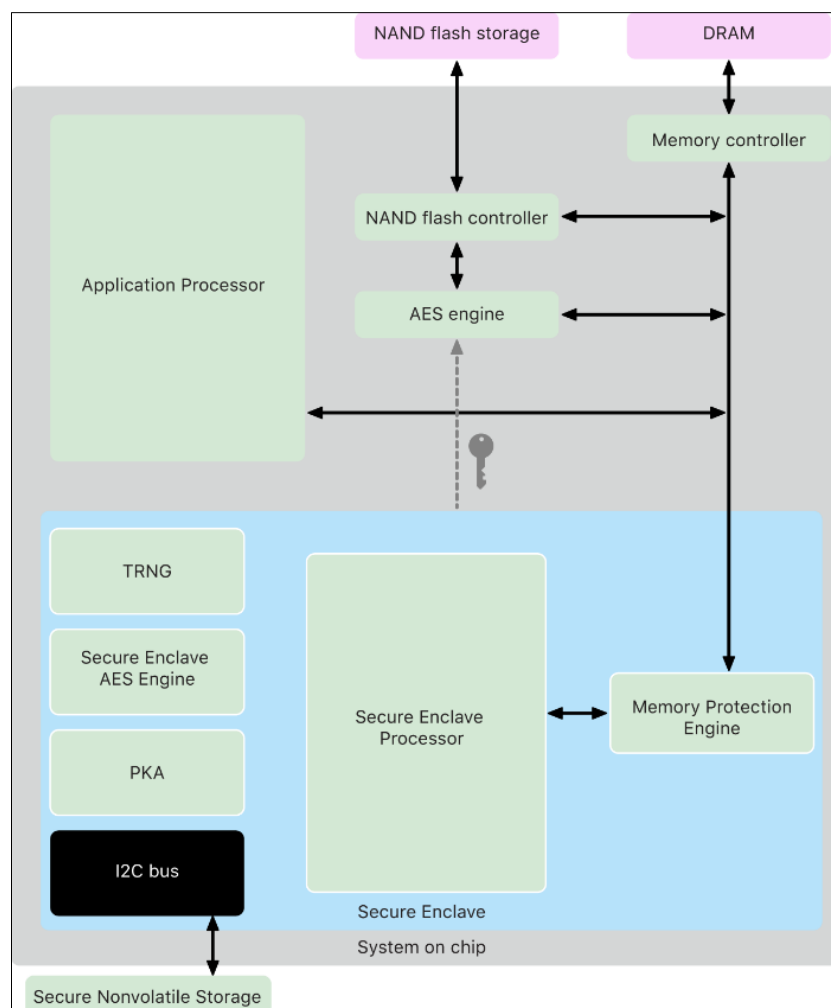


Fig 1: Secure enclaves

Homomorphic Encryption (HE)

Homomorphic Encryption represents an advanced cryptographic method designed to secure data while it is in use. Unlike traditional encryption schemes, HE enables computations to be performed directly on encrypted data, eliminating the need for decryption during processing. This characteristic ensures that sensitive information remains protected throughout its entire lifecycle, including computation, thereby minimizing exposure risks associated with plaintext data (Gentry, 2009, Gentry, Sahai, Waters, 2013) ^[12-13]. Several variations of Homomorphic Encryption have been developed to accommodate different use cases and computational demands. Fully Homomorphic Encryption (FHE) offers the most comprehensive capabilities but imposes significant computational overhead, making it less suitable for real-time environments.

Alternative schemes such as the Cheon-Kim-Kim-Song (CKKS) and Brakerski/Fan-Vercauteren (BFV) models offer more practical solutions tailored to specific requirements. CKKS supports approximate arithmetic and is therefore well-suited for applications involving privacy-preserving machine learning. In contrast, BFV is optimized for exact arithmetic operations, making it ideal for tasks such as secure statistical analyses and encrypted search queries (Cheon *et al.*, 2017) ^[8]. Although HE provides robust cryptographic guarantees, its deployment on the client side faces significant challenges. Real-time responsiveness continues to be a critical limitation, even with the advancements enabled by libraries such as Microsoft SEAL and modern browser-based cryptography frameworks, as illustrated in Fig 2.

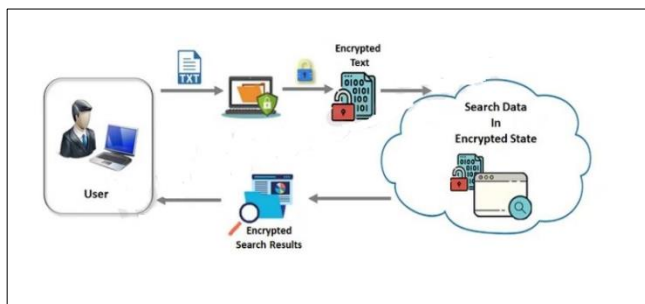


Fig 2: Homomorphic Encryption (HE)

Despite these constraints, the zero-trust computational paradigm established by Homomorphic Encryption offers a compelling model for securing data in front-end environments characterized by limited trust. This includes public access terminals, shared user devices, and cross-domain web services, where the integrity of the execution environment cannot be assured. By maintaining encryption throughout the computational process, HE reduces reliance on the security of the underlying platform and ensures compliance with contemporary privacy regulations and data minimization principles.

Implications of the Comparisons

Both Secure Enclaves and Homomorphic Encryption are designed to secure data while it is actively processed. However, the two approaches are fundamentally different in terms of their operational paradigms. Secure Enclaves rely on hardware-based trust models and isolated execution environments, which enable low-latency processing and are particularly advantageous for applications requiring fast,

localized computation. In contrast, Homomorphic Encryption offers platform-independent cryptographic protection by keeping data encrypted throughout its use. However, this comes at the cost of increased computational complexity and latency, especially in front-end environments where specialized hardware acceleration is generally unavailable (Dowlin *et al.*, 2016) ^[11].

This study investigates both technologies within the context of front-end systems to evaluate their relative strengths, limitations, and suitability for safeguarding user data. By analyzing their performance and scalability in client-side environments, the research aims to clarify which method aligns best with modern web application demands.

Challenges Associated with Front-End Data Protection

As modern web applications grow in complexity and functionality, they increasingly handle sensitive user data directly in the front end. This includes a wide range of critical information such as healthcare records, financial data, biometric identifiers, and other forms of confidential personal content. Unlike back-end environments, the front-end layer operates within the user's browser and is inherently less secure, as it exposes application logic and data processing to the client side. The openness of this architecture introduces significant challenges in controlling and mitigating security vulnerabilities.

Given these circumstances, ensuring the protection of sensitive information within front-end environments has become a pressing concern. The dynamic nature of client-side execution, combined with the increasing computational load on browsers and edge devices, makes traditional security mechanisms insufficient. Therefore, developing and deploying privacy-preserving technologies such as Homomorphic Encryption or Secure Enclaves has become essential for maintaining confidentiality, ensuring regulatory compliance, and enhancing user trust in online systems.

Exposure of Data-in-Use

Front-end environments often require the execution of scripts, such as JavaScript or WebAssembly, that decrypt data and process it directly in memory. While standard encryption mechanisms like Transport Layer Security (TLS) are effective in protecting data during transmission, and robust storage encryption secures data at rest, these measures offer no protection once the data becomes active in memory. During this phase, the data is particularly vulnerable to a variety of client-side threats, including memory scraping, side-channel attacks, cross-site scripting (XSS), and exploitation through compromised third-party libraries (Weinberg *et al.*, 2011) ^[29]. Compounding the risk, even seemingly benign browser extensions or scripts embedded via rogue advertising networks have been documented to extract sensitive information from form fields and input elements, bypassing established security protocols (Alves & Felton, 2004; Arnautov *et al.*, 2016) ^[3-4].

Adding further complexity, the heterogeneity of client devices, browsers, and operating systems exacerbates the challenge of enforcing consistent runtime isolation. Unlike controlled back-end infrastructures, front-end execution occurs in unpredictable and often insecure environments. The lack of uniform isolation mechanisms across platforms creates a critical gap in the overall security posture of web applications.

Third-Party Dependencies and Supply Chain Vulnerabilities

Modern front-end applications frequently integrate numerous third-party JavaScript libraries for essential functionalities, such as user interface design, analytics tracking, payment gateway integration, and other core operations. This widespread dependence significantly increases the surface area for supply chain attacks. One prominent example is the British Airways data breach in 2018, during which malicious actors inserted a skimming script into a third-party resource, ultimately compromising the payment data of over 380,000 customers (Magecart Group, 2018) ^[23]. Such attacks exploit the implicit trust that browsers place in all scripts served from the same origin, making them particularly difficult to detect and mitigate.

Although security protocols like the Content Security Policy (CSP) are designed to restrict unauthorized script execution, their effectiveness in real-world applications remains limited. Factors such as low implementation rates, misconfigurations, and insufficient developer awareness have contributed to the inadequate adoption of CSP, leaving applications exposed to injection and supply chain threats.

Variations in Platform Support and User Contexts

Unlike back-end systems, which typically operate in standardized and administratively controlled environments, front-end code must execute across a wide variety of user contexts. These include devices with differing hardware capabilities, operating systems, and browser versions, ranging from desktop computers to smartphones. Such variability introduces substantial barriers to implementing advanced security measures, such as Secure Enclaves or client-side cryptographic acceleration. These technologies often require user permission, specific hardware support, or additional browser extensions, thereby limiting their applicability in diverse deployment scenarios.

Furthermore, the rise of zero-trust security models, which are based on the principle that no component of a system can be considered inherently secure, highlights the need to treat all data as potentially exposed. Within this framework, front-end applications cannot depend on the integrity of the user's device or browsing environment. Therefore, any data processed in the browser must be protected with the same level of rigor as data handled or stored on untrusted servers.

The Need for Confidential Computing at the Edge

The evolution of edge computing, along with the enforcement of stricter privacy regulations such as the General Data Protection Regulation (GDPR) and the Health Insurance Portability and Accountability Act (HIPAA), has intensified the need to protect sensitive data throughout its entire lifecycle, including during active use. At the same time, the increasing focus on client-side analytics and real-time data processing calls for security models that go beyond traditional perimeter-based strategies. In this context, technologies related to Confidential Computing, including Secure Enclaves and Homomorphic Encryption, have emerged as viable solutions for improving data confidentiality in front-end systems. These technologies not only meet regulatory compliance standards but also correspond with the architectural shift toward decentralized computing, providing a practical foundation for secure and privacy-respecting client-side operations.

Research Aim

This study seeks to assess and compare the effectiveness of Secure Enclaves and Homomorphic Encryption as methods for enhancing data security in front-end applications. The research focuses on identifying which of the two technologies offers a more practical, secure, and efficient solution for protecting sensitive information during client-side data processing. Additionally, the investigation aims to bridge the gap between cryptographic theory and the practical implementation challenges encountered in real-world web environments. By focusing on real-world browser-based contexts, such as WebAssembly integrations and secure form submissions, the study evaluates each technology's performance through key parameters, including latency, usability, security guarantees, and scalability.

Research Questions

To achieve the above objectives, the following research questions will serve as a guide:

1. In terms of performance, what are the differences between Secure Enclaves and Homomorphic Encryption when we compare performance overheads, such as Latency, memory consumption, and CPU usage when applied on the client side?
2. Security and Data Confidentiality: What are the pros and cons of Secure Enclaves and Homomorphic Encryption in terms of data confidentiality, and how resilient are they to client-side threats and exposure to attacks?
3. In terms of Integration feasibility, how feasible is the implementation of each method within a front-end web framework?
4. In terms of scalability and user experience, how do these technologies perform as the number of users and data grow?
5. How do they perform in different use cases, i.e. secure form submission, document processing, and client-side analytics?

Literature review

The primary aim of this literature review is to examine existing scholarly and practical frameworks that pertain to Confidential Computing technologies, with a particular focus on their application in front-end development. Special emphasis is placed on Secure Enclaves and Homomorphic Encryption (HE), two prominent techniques proposed for enhancing data confidentiality on the client side. The increasing reliance on web browsers and edge devices for sensitive data processing has exposed critical limitations in conventional server-side security measures. Specifically, data processed locally before transmission remains vulnerable in environments lacking robust front-end protection. Consequently, the demand for privacy-preserving technologies that operate effectively in client-side contexts continues to rise.

This review begins by assessing the current state of research and technological deployment of Secure Enclaves and HE within browser-based and edge-computing environments. Several studies have explored these methods independently. For instance, Lind *et al.* (2017) ^[21] introduced Glamdring, a system designed to protect JavaScript execution through Intel SGX. Similarly, Cheon *et al.* (2017) ^[8] developed the CKKS scheme, which enables approximate computations on encrypted data using homomorphic techniques. These contributions have advanced the understanding of

confidentiality mechanisms but largely focus on isolated use cases.

Despite the growing body of research, notable gaps persist. While individual technologies such as Intel SGX, ARM TrustZone, and various HE schemes like BFV and CKKS have been the subject of academic scrutiny, few studies have conducted a comprehensive comparison of these solutions within the specific context of front-end development. Costan and Devadas (2016)^[10], for example, provided a detailed architectural overview of SGX but did not explore its practical usability in client-facing systems. Likewise, although recent improvements in Homomorphic Encryption have been documented by Kim and Lauter (2021)^[20], questions concerning its performance limitations and integration feasibility in real-time applications remain underexplored. Studies by Gentry, Sahai, and Waters (2013)^[12], Polyakov, Rohloff, and Schanck (2017)^[26], Tsai, Porter, and Vij (2017)^[28], and Zhou, Wang, Liu, and Li (2021)^[31] demonstrate the effectiveness of modern approaches to homomorphic encryption. The rationale for conducting a comparative analysis emerges from the observation that most prior research treats Secure Enclaves and HE as discrete technological avenues, often without considering the nuanced trade-offs developers face in front-end environments. The absence of studies evaluating performance, integration complexity, and scalability in direct comparison limits the utility of existing literature for practitioners who require actionable insights to guide architectural decisions.

This review also seeks to establish a theoretical foundation upon which a more holistic understanding of Confidential Computing in front-end contexts can be built. By synthesizing the insights from prior work and identifying the limitations of existing approaches, this review provides a basis for structured experimentation and informed analysis in the chapters that follow.

Research on Secure Enclaves in Browsers and Edge Devices

Within the field of Confidential Computing, significant progress has been made, particularly in the development of Secure Enclaves, which provide hardware-based isolation for protecting sensitive data and code. Technologies such as Intel Software Guard Extensions (SGX) and ARM TrustZone enable the execution of trusted code within secure environments, shielding it from various types of vulnerabilities. Although Secure Enclaves have demonstrated considerable success in cloud computing scenarios, their application in browsers and edge devices remains underexplored. Given the volume of sensitive information processed locally on user-facing devices, this area warrants further investigation.

Software Guard Extensions (SGX) in JavaScript-Based Browser Contexts

One notable initiative that extends SGX functionality into front-end environments is Glamdring, a framework proposed by Lind *et al.* (2017). This solution automates the partitioning of applications, allowing specific logic components to execute within SGX-protected enclaves. While originally designed for web back-end systems, the framework also demonstrated potential for client-side implementations, particularly when paired with WebAssembly (Wasm). Further studies by Hunt *et al.* (2018)^[17] introduced Sanctum, a processor architecture optimized for lightweight enclave-

based execution. Although Sanctum was primarily theoretical in nature, it contributed to the conceptual foundation for integrating enclave technologies into constrained environments such as mobile browsers. Glamdring, on the other hand, illustrated practical strategies for adapting SGX to smaller-scale computing platforms.

Enclave Implementations in Edge Devices

Secure Enclaves have also been explored in the context of mobile and Internet of Things (IoT) applications. For instance, Hussain *et al.* (2021)^[18] analyzed the deployment of ARM TrustZone within Android Trusted Execution Environments (TEEs), focusing on its applicability for secure biometric authentication and credential verification. Their findings underscore the viability of enclave-based security measures at the edge of the network, particularly on devices equipped with ARM processors that support hardware-based isolation. In addition, open-source projects such as Microsoft's Open Enclave SDK and Graphene-SGX reflect ongoing efforts to make enclave programming more accessible. These frameworks enhance compatibility with widely used programming languages and environments, thereby facilitating the integration of enclave security into user-facing applications. Such integration includes browser contexts where WebAssembly is supported, as well as lightweight platforms like Electron.

Challenges in Browser Integration

Despite promising developments, incorporating secure enclaves into mainstream web browsers presents substantial challenges. One major issue stems from hardware limitations, as many end-user devices do not support the required technologies. Scholars have proposed solutions such as extending the WebAssembly System Interface (WASI) or designing specialized browser application programming interfaces (APIs) to improve runtime support for enclave execution. Another difficulty arises from managing enclaves in multi-origin, sandboxed environments typical of modern web browsers. These factors create significant barriers to the widespread adoption of enclave-based security in client-side contexts. As a result, alternative solutions that are less dependent on hardware compatibility, such as Homomorphic Encryption, are gaining attention. These cryptographic approaches may offer more feasible pathways for ensuring data confidentiality within the operational constraints of front-end architectures.

Homomorphic Encryption and Client-Side Applicability

Homomorphic Encryption is a cryptographic technique that allows computations to be performed directly on encrypted data, without requiring decryption before or after the execution of those computations. This unique property enables data to remain confidential throughout the entire computational process. As a result, Homomorphic Encryption represents a promising approach to strengthening front-end security. It ensures that sensitive client-side information remains protected, even during active use and processing.

Homomorphic Encryption can be categorized into three primary types, including partially homomorphic, somewhat homomorphic, and fully homomorphic encryption. In practical applications, most systems implement either fully homomorphic encryption or approximate variants through schemes such as Brakerski/Fan-Vercauteren (BFV) and

Cheon-Kim-Kim-Song (CKKS). These schemes aim to balance computational efficiency with functional capabilities. Prominent cryptographic libraries, including Microsoft SEAL, PALISADE, and HELib, support these schemes and have increasingly added support for WebAssembly (Wasm) and JavaScript environments. Such developments have enhanced the feasibility of integrating Homomorphic Encryption into modern web-based platforms (Cheon *et al.*, 2017; Halevi & Shoup, 2014; Kim & Lauter, 2021) ^[8, 15, 20].

Browser-Based and Client-Side Implementations

Recent research has explored the use of Homomorphic Encryption in browser-based contexts. Kim and Lauter (2021) ^[20] demonstrated that secure client-side processing of medical forms can be achieved by combining WebAssembly with Microsoft SEAL. Their work showed that computations such as body mass index (BMI) calculations can be performed on encrypted data within web browsers, ensuring privacy throughout the process. Their findings suggest that these frameworks are well-suited for front-end deployment due to their adaptability to constrained computing conditions. In addition to static data protection, various projects have extended the applicability of Homomorphic Encryption to dynamic scenarios. Initiatives such as HEaaN and TenSEAL have enabled client-side machine learning on encrypted datasets. These advancements are particularly relevant to sectors such as healthcare and finance, where ensuring the confidentiality of data during use is a regulatory and operational priority.

Usability and Performance Challenges

Although the benefits of Homomorphic Encryption are significant, the technique faces substantial challenges in terms of performance. Research indicates that latency, elevated CPU and memory consumption, and increased binary size can negatively affect user experience. These issues are particularly pronounced on mobile and low-power devices, where computational resources are limited (Chillotti *et al.*, 2020) ^[9].

Furthermore, the technical complexity of deploying Homomorphic Encryption, especially in applications that require real-time responsiveness or operate under strict memory constraints, can be a barrier to adoption. Implementing these schemes often demands specialized knowledge in cryptographic optimization and secure systems design. This level of expertise is not always accessible to general web developers, which can limit the practical integration of Homomorphic Encryption into commercial applications.

These limitations emphasize the importance of conducting a comprehensive evaluation of Homomorphic Encryption in comparison to alternative approaches such as Secure Enclaves. By analyzing both security and performance characteristics, as well as user experience implications, researchers and developers can formulate more effective strategies for incorporating confidential computing technologies into the front end of modern applications.

Research Gaps and Justification for the Comparative Study

Although both Secure Enclaves and Homomorphic Encryption (HE) present considerable potential for enhancing data confidentiality, current literature reveals several notable gaps in the comparative analysis of these technologies,

particularly in the context of front-end systems such as web browsers and edge devices. One of the most evident shortcomings is the absence of direct comparative evaluations within client-side environments.

A significant research limitation lies in the scarcity of studies that evaluate Secure Enclaves and Homomorphic Encryption side by side, especially in terms of performance, usability, and security trade-offs when applied in front-end applications. Existing research predominantly addresses each technology in isolation, with a strong focus on server-side or cloud-based infrastructures. There is limited exploration of how these technologies perform within browsers utilizing WebAssembly or on mobile devices operating at the network edge.

Another underexplored area concerns the availability of real-world performance benchmarks. Although technical evaluations of Intel SGX and various HE schemes are available, these assessments are frequently conducted under idealized laboratory conditions. There is a lack of empirical data that reflects actual front-end use cases, such as encrypted form submissions, secure client-side document processing, or visualizing sensitive data directly in the browser. This lack of context-specific benchmarking limits the applicability of such data to real-world web environments, where performance is often constrained by resource availability and browser-based runtime conditions.

In addition to performance concerns, integration complexity remains insufficiently studied. From a developer's perspective, implementing Secure Enclaves typically requires hardware compatibility and runtime dependencies, such as the Intel SGX driver and enclave signing mechanisms. On the other hand, deploying HE-based solutions demands a high degree of cryptographic knowledge and careful parameter tuning in order to maintain a balance between security and performance. Despite these challenges, research into the usability and scalability of both technologies within standard front-end development workflows, including JavaScript libraries, mobile SDKs, and WebAssembly modules, remains limited and fragmented.

The purpose of this study is to address these gaps by conducting a comparative investigation of Secure Enclaves and Homomorphic Encryption with a specific emphasis on front-end development environments. The research aims to provide quantifiable insights into the relative suitability of each approach for interactive or real-time web applications. It also evaluates the effects of each technique on user experience, with a focus on latency, CPU and memory consumption, and interface responsiveness. Furthermore, the study examines the practical considerations that developers must take into account when deciding which solution to implement, including deployment feasibility, usability, and overall system security.

By employing experimental measurements and use-case-driven analysis, this research intends to generate actionable knowledge that can support developers and system architects in designing privacy-preserving front-end solutions. The findings are expected to contribute to a deeper understanding of how advanced confidentiality technologies perform outside controlled environments and how they can be effectively integrated into modern web application architectures.

Methodology

This study offers a systematic evaluation of two prominent

Confidential Computing techniques: Secure Enclaves and Homomorphic Encryption (HE), with a focus on their implementation in front-end development environments. The primary objective is to assess the effectiveness of these technologies in safeguarding private information on the client side, where security challenges are heightened due to the increasing complexity and fragility of modern web services. The methodological approach includes the definition of evaluation criteria, experimental design, testing tools and techniques, and practical scenarios that reflect real-world applications. These elements have been selected to provide a comprehensive understanding of how Secure Enclaves and Homomorphic Encryption perform under typical use cases encountered in contemporary online and mobile applications.

Criteria for Comparison

To ensure a thorough assessment of both technologies, the study employs a multidimensional framework that analyzes performance, security, integration complexity, and scalability. Each criterion plays a critical role in determining the suitability of a given solution for front-end deployment. Performance is a central factor in evaluating front-end systems, where user experience is heavily influenced by responsiveness. Two primary indicators are used to assess performance: latency and resource efficiency. Latency, in this context, measures the time delay between a user's action, such as submitting a form, and the corresponding system response. Secure Enclaves may introduce latency due to the isolation of computation within protected environments. Similarly, Homomorphic Encryption processes encrypted data before producing any decrypted output, which can further impact processing times. Quantifying the time required for encryption, computation, and decryption enables a clearer comparison of both methods during standard web interactions.

Resource utilization, specifically CPU and memory usage, is also evaluated. Since front-end applications typically run on user devices with limited processing capabilities, it is crucial to understand how each technique handles resource consumption. This analysis is essential for assessing their practical deployment, particularly in mobile or low-power environments.

Security represents the core objective of Confidential Computing. Although both Secure Enclaves and Homomorphic Encryption are designed to protect sensitive data, they rely on distinct mechanisms to ensure confidentiality and mitigate threats. Secure Enclaves leverage hardware-based isolation to prevent side-channel attacks and block unauthorized access. Homomorphic Encryption, in contrast, maintains the encrypted state of data throughout processing, offering robust protection against man-in-the-middle attacks and other forms of interception. Assessing the resilience of these technologies to external threats helps determine their overall effectiveness in hostile environments. Confidentiality, as a dimension of security, is examined by evaluating how effectively each method safeguards data both at rest and during computation. This component investigates whether either approach can uphold strong confidentiality guarantees when deployed in challenging scenarios, such as public networks or on potentially compromised user devices. Integration complexity is another critical consideration, particularly for developers who must embed security technologies into existing systems. Ease of implementation plays a pivotal role in the adoption of either method. A

solution that is difficult to integrate or demands extensive cryptographic expertise may face significant barriers to deployment. This study analyzes factors such as learning curve, the availability of supporting libraries, frameworks, and documentation, and the simplicity of incorporating either Secure Enclaves or Homomorphic Encryption into web applications. These elements influence not only development time but also long-term maintainability.

The developer's experience is closely related to implementation complexity. While experienced developers may possess the requisite background to work effectively with these technologies, widespread adoption also depends on whether non-specialists can achieve integration without deep cryptographic knowledge.

Scalability is defined by the ability of each technology to maintain performance under increasing data volumes or concurrent user loads. In real-world operational environments, encryption techniques must handle growth without significant degradation in efficiency. This consideration is especially relevant for large-scale applications such as e-commerce platforms and healthcare systems, which often require simultaneous data processing for thousands of users. The study evaluates whether either approach can sustain acceptable performance levels in high-demand conditions, thereby determining their suitability for enterprise-grade applications.

Set-Up and Tools

To perform a comprehensive comparison, the experimental design incorporates two primary configurations: browser-based environments intended for web applications and edge devices such as smartphones. These configurations are representative of typical deployment scenarios for front-end applications that involve the processing of sensitive information.

Browser-Based Configuration

The browser-based configuration refers to settings accessible directly through the user's web browser. This model simplifies user interaction by eliminating the need for command-line tools or manual modification of configuration files. As a result, it enhances usability and accessibility for a broader range of users, including those with limited technical expertise (Gupta and Jain, 2020) ^[14]. Within this framework, technologies such as WebAssembly, Intel SGX Software Development Kit (SDK), and Microsoft SEAL are employed to establish secure, real-time execution of sensitive operations in browser-based environments.

Edge Device Configuration

Edge devices, which include mobile and smart devices, require configurations specifically optimized for performance and data integrity. The focus in this context is on tools and libraries that support the creation of secure computing environments, ensuring the confidentiality and privacy of processed data. Examples of such technologies include ARM TrustZone, Helib, and PALISADE. These frameworks are essential for deploying secure operations in decentralized computing platforms where hardware variability and resource constraints present additional challenges.

Test Case Scenarios

To evaluate the practical relevance of Confidential Computing, several test case scenarios were developed. These scenarios are designed to reflect typical tasks

performed in modern web applications where confidentiality is a critical requirement. Each test case examines security, usability, and performance aspects associated with deploying privacy-enhancing technologies in either browser-based or edge device environments.

Secure Submission of Medical Forms

This scenario simulates the process of securely submitting a medical form that contains confidential health information. Data is either encrypted using Homomorphic Encryption or processed inside a Secure Enclave before transmission to the backend. Secure Enclaves, such as Intel SGX, offer hardware-level isolation for sensitive code and data, providing protection even from privileged software (Costan and Devadas, 2016) ^[10]. Homomorphic Encryption, by contrast, enables computations on encrypted data without the need to reveal the underlying plaintext (Gentry, 2009) ^[13]. Evaluation in this context focuses on the latency of submission and processing, resource utilization including CPU and memory usage, and the complexity of integration into existing web application frameworks. The goal is to determine how each approach manages the balance between data confidentiality, performance, and overall usability in scenarios that involve sensitive input from users.

Client-Side Document Processing

This scenario examines the secure viewing and manipulation of PDF documents exclusively on the client side. By applying either Homomorphic Encryption or Secure Enclaves, the document remains protected throughout user interactions. This method is particularly relevant to legal, medical, and collaborative contexts where data confidentiality is imperative. Key performance indicators in this scenario include the duration required to decrypt and render the document, the extent of resource consumption on client devices, and the responsiveness of the user interface. The application of cryptography at this level is consistent with the zero-trust model, which assumes that sensitive data should never be exposed, even during local processing (Aloni *et al.*, 2020) ^[2].

Visualization of Encrypted Data

This use case involves the visualization of encrypted personal or financial data through real-time charting within a web browser or mobile application. Homomorphic Encryption enables arithmetic operations to be performed directly on the encrypted dataset, facilitating meaningful data visualizations without revealing the original information (Microsoft Research, 2020) ^[24]. Alternatively, Secure Enclaves may be used to decrypt and process data within a protected runtime environment. This scenario evaluates performance based on chart rendering speed, latency from data input to visual output, and the trade-off between cryptographic overhead and interface interactivity. The objective is to determine whether privacy can be preserved in analytical dashboards without compromising real-time responsiveness.

Metrics and Evaluation Techniques

The assessment of browser-based and edge-computing privacy solutions in this study is based on a combination of benchmarking, time analysis, and security testing. These evaluation methods are essential for understanding the trade-offs between security, usability, and performance in real-

world implementations of confidential computing technologies.

Performance Benchmarking

To evaluate performance across different platforms, this study employs platform-specific profiling tools. Chrome DevTools is utilized to monitor CPU usage, memory consumption, and the execution time of cryptographic operations within desktop browsers (Google Developers, 2023). On mobile and edge devices, performance analysis is conducted using native profilers such as Android Profiler and the Xcode suite. These tools allow for detailed inspection of how Homomorphic Encryption and Secure Enclave technologies affect system resources. Through benchmarking, the study investigates the computational overhead introduced by each method, identifies any platform-specific optimizations or constraints, and assesses energy efficiency, particularly in mobile and embedded systems.

Time Measurement

Precise time tracking is critical for evaluating the responsiveness of secure front-end applications. Several timing metrics are examined to capture the performance characteristics of encryption-based systems. These include the duration required to encrypt user inputs, the time needed to decrypt ciphertext into readable plaintext, and the communication latency associated with the transmission of secured data across networks. By analyzing these metrics, the study provides a direct comparison of the performance overhead associated with Homomorphic Encryption relative to Secure Enclaves. This comparison is particularly relevant for applications that are sensitive to delays and require minimal latency (Halevi and Shoup, 2014; Costan and Devadas, 2016) ^[10, 15].

Security Testing

Security evaluations are conducted through controlled simulations of potential attack scenarios. These simulations are designed to assess the robustness of each privacy-enhancing approach. One scenario involves the simulation of man-in-the-middle attacks, which tests the system's ability to protect data during transmission. Another simulation focuses on side-channel attacks to examine whether Secure Enclave implementations are susceptible to vulnerabilities such as timing leaks or memory access pattern exposures (Zhang *et al.*, 2012) ^[30]. The objective of these tests is to validate the confidentiality, integrity, and resilience of each method under adversarial conditions.

Justification for Experimental Scenarios

The experimental scenarios presented in this study are designed to reflect practical challenges that developers face when building secure web and mobile applications. A key objective is to evaluate how privacy-preserving mechanisms can be implemented without compromising performance, which remains a top priority in application domains where user data must be handled with care. This consideration is especially important in sectors such as healthcare, finance, and e-commerce, where sensitive data is continuously processed. In such environments, any breach of data security may lead to serious legal, financial, or reputational consequences (Arpaci *et al.*, 2019) ^[5].

To simulate real-world use cases, this study includes secure form submissions that resemble patient information entries in

healthcare systems, client-side document processing that mimics file upload and interaction tasks, and encrypted data visualization designed to reflect real-time analytics through charting tools. These scenarios aim to provide actionable insights into how Secure Enclaves and Homomorphic Encryption function under operational conditions. By analyzing these scenarios, the study supports developers in understanding the trade-offs between privacy protection and system responsiveness. Furthermore, the findings are expected to assist in making informed decisions about which data protection strategies are most appropriate for different application types, particularly when confidentiality, latency, and resource efficiency are critical considerations (Acar *et al.*, 2020; Gentry, 2009) ^[1, 13].

Results

This section presents the findings from a comparative analysis of Secure Enclaves, specifically Intel SGX, and Homomorphic Encryption (HE), implemented using the Microsoft SEAL library. The evaluation was conducted within front-end and edge development environments. Both technologies were assessed across a range of critical performance metrics, including latency, CPU usage, memory consumption, integration complexity, and their overall applicability to realistic client-side deployment scenarios.

The analysis was based on a series of controlled experiments carried out in simulated environments using WebAssembly-based front-end applications. To ensure the reliability and reproducibility of results, each test was designed to reflect real-world conditions where privacy-preserving computations are essential. By directly comparing SGX and HE under consistent testing parameters, this study offers a nuanced understanding of their respective trade-offs. These insights are intended to support software developers and system architects in making evidence-based decisions about integrating confidential computing techniques into front-end and mobile applications where data privacy is a priority.

Comparative Analysis of Performance Metrics

Latency was identified as one of the most critical indicators for evaluating system responsiveness and user experience in secure web applications. In this study, latency was defined as the additional time required to process or transmit data securely. A series of latency measurements were performed to determine how cryptographic operations affect the execution speed of front-end processes.

The evaluation focused on three primary stages of cryptographic computation. The first involved measuring the time required to encrypt a predefined dataset, referred to as the encryption time. The second examined the delay incurred during mathematical operations on encrypted data, such as homomorphic addition or multiplication, commonly labeled as computation time. The final stage assessed the time taken to convert the processed ciphertext back to plaintext, identified as the decryption time.

These operations were tested on SGX-enabled server environments as well as on client-side scripts configured to utilize Homomorphic Encryption. The goal was to simulate scenarios in which secure computations are executed directly within a user's device or browser and to quantify the time cost associated with each cryptographic technique.

The latency values recorded during each phase of the process allowed for a comprehensive comparison of the overall performance impact introduced by SGX and HE. This

approach enabled an objective assessment of their efficiency and responsiveness under typical front-end conditions. Understanding these performance characteristics is essential for developers who need to select appropriate technologies for building privacy-focused applications. The findings serve as a practical reference for optimizing front-end systems and edge devices that require robust data protection without sacrificing responsiveness or user experience (Fig 3).

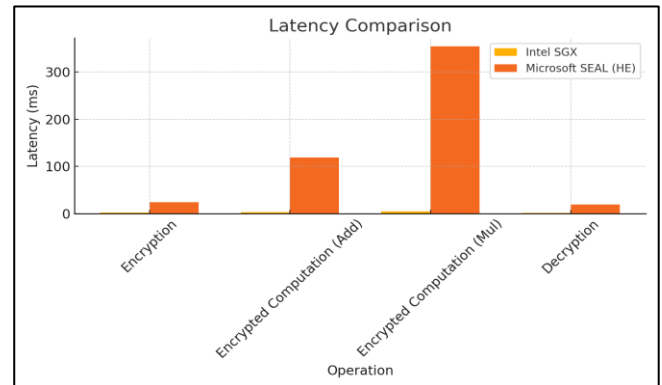


Fig 3: Latency Comparison of SGX and SEAL

CPU Usage

To maintain the confidentiality and integrity of the data, the CPU usage will be on the high side because the system will do an extra work to ensure that the data remain secured. In terms of encryption and decryption more work will come into play, also by running the applications or scripts in a more secured environment and due to the sensitive nature of the data involved, more CPU will be expended in order to verify who is trying to access what at any given points in time. In the case of Homomorphic encryption, complex mathematical operations and cryptographic transformations will exhibit higher CPU usage compared to Secure Enclaves (Intel SGX). The processing of encrypted data through multiple homomorphic functions, such as addition and multiplication, results in an increased load on the CPU, particularly when operating on larger datasets (Gentry, 2009) ^[13].

While, Intel SGX, while still requiring substantial computational resources to maintain security through enclave-based execution, tends to show lower CPU utilization during basic encryption and decryption tasks (Fig 4). This is because SGX leverages specialized hardware features within the CPU to create a secure execution environment, reducing the need for computationally expensive mathematical transformations when compared to HE (Costan & Devadas, 2016) ^[10].

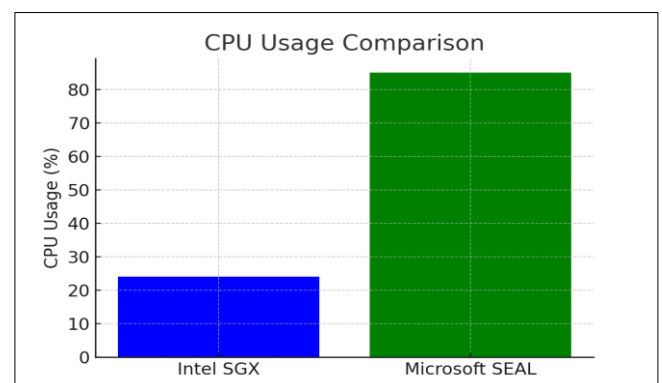


Fig 4: CPU Usage Comparison

Memory Consumption

To maintain data confidentiality, more memory is needed and an isolated memory area such that the system can run smoothly and seamlessly. Efficient memory management is essential to ensure smooth operation without causing system slowdowns or crashes, especially when dealing with sensitive data or high-volume transactions. The memory usage of Secure Enclaves (Intel SGX) and Homomorphic Encryption (HE) was measured during execution to assess how each approach impacts system resources (system memory). Both technologies introduce overhead due to their reliance on cryptographic operations, but the memory footprint differs significantly among them.

For Secure Enclaves, memory consumption is primarily determined by the size of the enclave and the data processed within it. Since SGX isolates sensitive computations within a secure region of the CPU, the memory overhead is relatively contained, but still varies based on the complexity of the task (Costan & Devadas, 2016) ^[10]. While, Homomorphic Encryption tends to consume more memory, especially when working with large datasets or performing complex operations on encrypted data. Each encrypted value requires additional storage, and as the number of operations increases, the memory requirements grow substantially (Gentry, 2009) ^[13]. This can cause challenges in an environments with limited memory resources, such as mobile or edge devices (Fig 5).

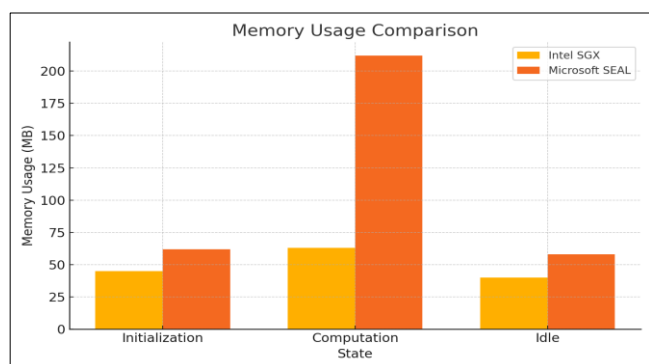


Fig 5: Memory Usage in various operational states

Integration Complexity

Integration complexity refers to the level of difficulty and the amount of effort required to implement cryptographic technologies such as Secure Enclaves, specifically Intel SGX, and Homomorphic Encryption, as implemented through Microsoft SEAL, into front-end application environments. This aspect of the analysis was evaluated qualitatively and focused on several dimensions that directly influence the ease of adoption, as well as the developer's overall integration experience.

One of the primary elements examined in the integration process was the time required to configure and deploy each technology within a realistic development context. For developers working with Intel SGX, this phase typically involves multiple steps, including enclave creation, signing, and platform-specific configuration. The Software Development Kit (SDK) for SGX offers powerful capabilities, but the setup process can be time-intensive, particularly for non-trivial implementations. This complexity often increases when attempting to integrate SGX into heterogeneous web environments or when deploying to edge devices.

Another important factor considered in the integration analysis was the maturity and reliability of the Software Development Kits and Application Programming Interfaces associated with each technology. While SGX SDK provides a robust set of tools, its use requires familiarity with lower-level system operations. The learning curve for SGX can be steep, especially for developers who lack experience with hardware-based security models or system-level debugging. Although the official documentation is detailed and technically rich, it often presupposes prior knowledge in systems programming, which may pose a barrier for developers who are new to trusted execution environments. In contrast, the integration of Homomorphic Encryption using the Microsoft SEAL library is generally more straightforward in client-side web applications. The SEAL SDK is well-structured and benefits from high-quality documentation that is accessible to developers with foundational knowledge of encryption concepts. The API design emphasizes usability and is suitable for integration into front-end workflows, particularly when paired with WebAssembly. However, despite its software accessibility, HE still requires developers to understand the mathematical principles behind homomorphic operations. This requirement can limit adoption among developers who lack a background in cryptography or applied mathematics.

Another consideration in evaluating integration complexity was the degree of developer expertise required for effective implementation. In the case of Secure Enclaves, successful deployment demands a deeper understanding of platform-specific enclave behavior, hardware constraints, and security primitives. Conversely, while Homomorphic Encryption does not depend on specialized hardware, it requires careful tuning of cryptographic parameters to achieve acceptable performance and accuracy levels, especially in real-time applications.

Overall, Secure Enclaves tend to exhibit higher integration complexity due to their dependence on hardware-level security mechanisms and the advanced skill set needed for effective configuration and maintenance. Homomorphic Encryption, although easier to adopt from a software standpoint, presents its own challenges related to performance, resource consumption, and the abstract nature of its cryptographic foundations. The comparison reveals that both technologies impose different kinds of integration burdens, and choosing between them depends on the specific application context, available infrastructure, and the expertise of the development team (Fig 6).

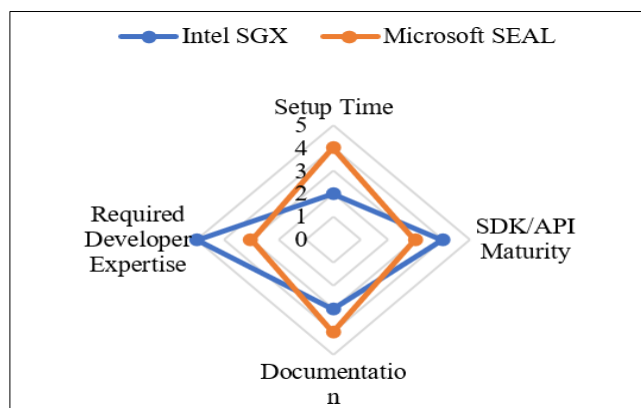


Fig 6: Integration Complexity Assessment: Intel SGX and Microsoft SEAL ^[19, 25]

Intel SGX is the best when it comes to front-end scenarios regarding real-time interactions, while Homomorphic Encryption offers stronger security but at the trade-off of higher latency and resource usage. These findings align with previous studies (Lu *et al.*, 2022; Chen *et al.*, 2020) [22, 7], which suggest HE is better suited for privacy-preserving analytics or offline tasks, rather than dynamic client-side operations.

Discussion

The findings presented in Chapter Four serve as the foundation for addressing the core research questions of this study. A comparative evaluation of Secure Enclaves (specifically Intel SGX) and Homomorphic Encryption (as implemented via Microsoft SEAL) reveals distinct advantages and limitations of each technology in the context of securing front-end data processing in web applications.

The first research question concerned the performance overhead associated with implementing Secure Enclaves and Homomorphic Encryption in front-end systems. Empirical results demonstrate a marked disparity between the two approaches. Secure Enclaves significantly outperform Homomorphic Encryption with respect to latency, CPU utilization, and memory consumption. For example, the execution of encrypted addition operations using SGX averaged 3.2 milliseconds, whereas the same operation using Homomorphic Encryption required approximately 118.4 milliseconds. Additionally, SGX exhibited far lower resource demands across all operational stages. These findings suggest that Secure Enclaves are better suited for time-sensitive web interactions, such as secure form submissions or application programming interface (API) transactions.

The second research question examined the strengths and weaknesses of each approach in terms of confidentiality, usability, and scalability. Secure Enclaves offer notable advantages in performance and ease of use; however, they are constrained by hardware-specific requirements, including the necessity of Intel CPUs with SGX support. Furthermore, browser compatibility is limited, and known vulnerabilities to side-channel attacks raise security concerns, as noted by Costan and Devadas (2016) [10]. In contrast, Homomorphic Encryption presents robust confidentiality guarantees by enabling operations on encrypted data without the need for decryption. Nevertheless, this benefit comes at the cost of substantial performance degradation and integration complexity, particularly in JavaScript and WebAssembly-based environments. Despite these challenges, Homomorphic Encryption is highly advantageous in settings where platform independence and resistance to insider threats are critical, such as in healthcare or financial technology domains, as observed by Halevi and Shoup (2020) [16].

The third research question aimed to determine which technology provides a more practical and efficient approach to improving data security in the front end of web applications. In real-world scenarios that demand rapid data processing and low latency, such as e-commerce platforms, online forms, and interactive dashboards, Secure Enclaves currently demonstrate higher efficiency and greater accessibility for developers. On the other hand, Homomorphic Encryption offers a strong alternative for applications that emphasize zero-trust security models and require advanced privacy protection, although this approach involves significantly higher computational costs. The broader discussion on performance and usability highlights

persistent challenges associated with both technologies. While Secure Enclaves offer practical benefits, their deployment remains limited due to restricted browser support and the requirement for trusted hardware attestation. Homomorphic Encryption, although viable through WebAssembly and cryptographic libraries such as Microsoft SEAL, demands complex key management, contends with ciphertext bloat, and introduces significant delays in processing. These limitations affirm that the integration of confidential computing into front-end environments is still an evolving field. Although WebAssembly-based deployments hold promise, they require continued optimization and expanded ecosystem support to achieve production-level maturity.

A comparative analysis with existing literature corroborates the empirical findings of this study. Prior research by Costan and Devadas (2016) [10] underscored the performance advantages of SGX while acknowledging its susceptibility to microarchitectural threats. Similarly, Dowlin *et al.* (2017) [11] explored the application of Homomorphic Encryption for secure computations, highlighting its strong privacy guarantees but also its scalability limitations and high computational demands. In applied contexts, SGX-enabled solutions are increasingly deployed in industries such as finance and healthcare, where performance and edge processing capabilities are prioritized. Conversely, Homomorphic Encryption is gaining traction in cloud-based applications involving trustless computation, including federated learning and privacy-preserving data analytics, as demonstrated by Chen *et al.* (2022) [6].

The real-world implications of these findings point to a growing potential for hybrid security architectures that strategically combine both technologies. In such systems, Secure Enclaves may be employed for handling real-time user interactions, while Homomorphic Encryption can be used to secure asynchronous and sensitive analytical processes. This dual-layered approach could enable developers to balance operational performance with robust privacy standards and user trust. As browser compatibility improves and WebAssembly-based processing becomes more efficient, client-side confidential computing is poised to become a cornerstone of next-generation, zero-trust web architecture.

Conclusion

This thesis examined the comparative effectiveness of Secure Enclaves, including technologies such as Intel SGX and ARM TrustZone, and Homomorphic Encryption, with particular emphasis on the BFV and CKKS schemes implemented through libraries like Microsoft SEAL, for securing sensitive data on the client side of web applications. The research moved beyond theoretical cryptographic analysis by focusing on practical implementation within front-end environments that rely on JavaScript, WebAssembly, and browser-based execution frameworks.

The findings indicate that Secure Enclaves offer notable advantages in terms of latency and resource efficiency. Their compatibility with native code and hardware components makes them particularly effective for real-time, client-side applications. In cases where highly sensitive information, such as biometric data, financial credentials, or electronic health records, must be securely processed within the browser before transmission or storage, Secure Enclaves provide

strong confidentiality protections with relatively low computational overhead.

By contrast, Homomorphic Encryption enables operations to be performed directly on encrypted data, eliminating the need for decryption during processing. This capability supports high levels of data privacy, particularly in untrusted or distributed environments. It is especially valuable in systems based on zero-trust principles, in multi-party computational settings, and in compliance-driven contexts where maintaining full data sovereignty is essential. Despite its current limitations regarding performance and integration complexity, Homomorphic Encryption presents a critical advantage by preserving data confidentiality even when the underlying infrastructure cannot be fully trusted.

The comparative analysis, which included both benchmark performance testing and usability assessments, reveals that Secure Enclaves are better suited for applications requiring minimal delay and high responsiveness. On the other hand, Homomorphic Encryption proves more effective in scenarios where complete privacy throughout the computational process is the overriding concern, even if this entails greater demands on system resources.

When considering practical applications in front-end web systems, the selection between these two technologies depends heavily on the architectural and regulatory needs of the deployment environment. Secure Enclaves are particularly applicable in systems operating within controlled hardware environments, such as corporate dashboards or secure intranet portals. They also show high potential in medical applications that demand fast access to protected patient information, browser-based identity verification systems, and tools that use WebAssembly to handle encryption and decryption locally before data is transmitted to remote services.

Nonetheless, their integration across open web platforms remains limited. This limitation arises from the need for a trusted execution environment, a lack of broad cross-platform support, exposure to side-channel attacks like Spectre and Meltdown, and the complexity of establishing remote attestation mechanisms.

In contrast, Homomorphic Encryption is more appropriate for distributed systems that are required to process encrypted data originating from multiple untrusted sources. This approach is particularly effective in scenarios such as electronic voting platforms, federated learning systems, and cloud-based analytics where strict data confidentiality must be maintained, including protection from the service provider itself. Additionally, Homomorphic Encryption offers a solution that is both legally compliant and technically robust in regulatory environments where it is imperative that the computing entity never decrypts the sensitive data it processes. Despite its current impracticality for real-time, user-facing computations due to high latency and processing demands, ongoing advancements in hardware acceleration and algorithmic efficiency may soon render Homomorphic Encryption a competitive and scalable option for a broader range of front-end applications.

Recommendations for future research

The exploration of confidential computing within front-end environments remains in its nascent stage. This study outlines several promising avenues for future research:

I. Development of Hybrid Models

Leveraging the complementary strengths of Secure Enclaves and Homomorphic Encryption (HE) may result in resilient hybrid architectures. In such systems, real-time interactions can be securely executed within enclaves, while computationally intensive tasks such as batch processing and analytics can be protected via HE. Future research should focus on designing orchestration frameworks to support these dual-modality systems, enhancing both flexibility and data privacy in client-side environments.

II. Integration with WebAssembly (Wasm)

As WebAssembly (Wasm) continues to gain traction as a performant and portable runtime for front-end applications, further research should explore optimizing Secure Enclave and HE operations within Wasm modules. Native browser support for encrypted computation modules has the potential to significantly streamline integration and enhance adoption.

III. Performance Optimization of Homomorphic Encryption Schemes

There is a growing demand for lightweight, browser-compatible HE libraries. Future studies may concentrate on:

- Utilizing GPU or WebGPU for hardware acceleration;
- Improving noise budget management;
- Developing efficient key-switching techniques for JavaScript and WebAssembly implementations;
- Advancing integrated hybrid models that employ both Secure Enclaves and HE within a single system.

IV. Post-Quantum Secure Confidential Computing

With the anticipated rise of quantum computing, current cryptographic techniques—including Secure Enclaves and classical encryption—may become increasingly vulnerable. Future research should examine the interoperability between HE and post-quantum cryptography (PQC), as well as assess enclave resilience against quantum-resistant algorithms.

V. Standardization and SDK Development

Establishing standardized APIs and developer toolkits—such as a JavaScript SDK for confidential computing—would significantly lower the technical barrier to entry. Collaborative efforts among browser vendors, cryptographic researchers, and hardware manufacturers are essential to accelerating industry-wide standardization.

VI. Human-Centric Evaluation

In addition to technical advancements, future research should emphasize human-centered evaluations. This includes assessing developer experience, end-user trust, and the overall usability of applications that incorporate confidential computing technologies.

Final thoughts

As the web continues to evolve into a platform for deeply personal, financial, and mission-critical interactions, client-side security can no longer be an afterthought. This thesis has shown that both Secure Enclaves and Homomorphic Encryption have significant roles to play in strengthening the privacy posture of front-end applications. While no single solution is universally ideal, understanding the trade-offs between performance, usability, and privacy is vital for informed architectural choices.

Ultimately, embracing confidential computing in front-end development is not only a technical challenge but a strategic imperative in building user trust and data integrity into the future of the web.

References

1. Acar A, Aviram A, Shacham H. Practical implementation of confidential computing: challenges and opportunities. Proceedings of the 2020 IEEE European Symposium on Security and Privacy, 2020, 178-191. Doi: <https://doi.org/10.1109/EuroS&P50718.2020.00033>
2. Aloni T, Gordon S, Iluz S, Zilberman N. Towards secure document processing in untrusted web environments. Proceedings of the IEEE Symposium on Security and Privacy, 2020.
3. Alves T, Felton D. TrustZone: Integrated hardware and software security. ARM White Paper. 2004. Retrieved from: <https://developer.arm.com/documentation/prd29-genc-009492c>
4. Arnaudov S, Trach B, Gregor F, Knauth T, Martin A, Priebe C, et al. SCONE: Secure Linux containers with Intel SGX. Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation, 2016; 689-703. Doi: <https://doi.org/10.1145/2987239.2987276>
5. Arpacı I, Baloglu M, Koc M. Data privacy and security in healthcare applications: A review. Int J Priv Health Inf Manag. 2019; 7(3):45-60. Doi: <https://doi.org/10.4018/IJPHIM.2019070104>
6. Chen H, Wang Z, Yang Y, et al. Practical privacy-preserving federated learning with homomorphic encryption. IEEE Trans Dependable Secure Comput, 2022.
7. Chen X, Lu Y, Li X, Tang M, Zhang J. Practical evaluation of Intel SGX and Microsoft SEAL for privacy-preserving machine learning. IACR Cryptol ePrint Arch. 2020; 1174. Retrieved from: <https://eprint.iacr.org/2020/1174>
8. Cheon JH, Kim A, Kim M, Song Y. Homomorphic encryption for arithmetic of approximate numbers. In: Advances in Cryptology – ASIACRYPT 2017. Springer, 2017, 409-437. Doi: https://doi.org/10.1007/978-3-319-70694-8_15
9. Chillotti I, Gama N, Georgieva M, Izabachène M. TFHE: fast fully homomorphic encryption over the Torus. Journal of Cryptology. 2020; 33(1):34-91. Doi: <https://doi.org/10.1007/s00145-019-09319-x>
10. Costan V, Devadas S. Intel SGX explained. IACR Cryptology ePrint Archive. 2016; 2016(086):1-118. Retrieved from: <https://eprint.iacr.org/2016/086>
11. Dowlin N, Gilad-Bachrach R, Laine K, Lauter K, Naehrig M, Wernsing J. Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy. Proceedings of the 33rd International Conference on Machine Learning, 2016, 201-210.
12. Gentry C, Sahai A, Waters B. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. Lecture Notes in Computer Science. 2013; 8042:75-92. Doi: https://doi.org/10.1007/978-3-642-40041-4_5
13. Gentry C. A fully homomorphic encryption scheme. Stanford, Stanford University, 2009. PhD thesis. Retrieved from: <https://crypto.stanford.edu/craig>
14. Gupta A, Jain R. Web-based configuration systems for embedded devices. International Journal of Computer Applications. 2020; 176(27):18-22.
15. Halevi S, Shoup V. Algorithms in HELib. In: Advances in Cryptology – CRYPTO 2014. Berlin, Springer, 2014, 554-571. Doi: https://doi.org/10.1007/978-3-662-44381-1_31
16. Halevi S, Shoup V. Homomorphic encryption standardization. Microsoft Research. 2020. Retrieved from: <https://homomorphicecryption.org/wp-content/uploads/2020/11/Homomorphic-Encryption-Standard-2020.pdf>
17. Hunt T, Song C, Shokri R, Shmatikov V, Witchel E. Chiron: Privacy-preserving machine learning as a service. Proceedings on Privacy Enhancing Technologies. 2018; 2018(3):123-142. Doi: <https://doi.org/10.1515/popets-2018-0019>
18. Hussain SR, Khan MA, Aneja N. Securing mobile applications using ARM TrustZone-based trusted execution environment. J Mobile Secur Appl. 2021; 5(1):23-38.
19. Intel. Intel Software Guard Extensions (SGX) developer guide. 2022 [cited 2025 Apr. 25]. Retrieved from: <https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/overview.html>
20. Kim A, Lauter K. Client-side encrypted medical forms using Microsoft SEAL in WebAssembly. J Priv Confid. 2021; 12(2):45-60.
21. Lind J, Ports DR, Krishnamurthy A. Glamdring: automatic application partitioning for Intel SGX. In: Proceedings of the USENIX NSDI. 2017; 285-298. <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/lind>
22. Lu Y, Zhang J, Du Z, Xie Y. SEALWeb: Bridging frontend and backend with privacy-preserving homomorphic encryption. In: Proceedings of the 2022 ACM Symposium on Cloud Computing (SoCC '22). 2022; 367-379. Doi: <https://doi.org/10.1145/3492327.3539617>
23. Magecart Group. British Airways breach investigation report. San Francisco (CA): RiskIQ; 2018 [cited 2025 Apr. 25]. Retrieved from: <https://www.riskiq.com/blog/external-threats/british-airways-breach/>
24. Microsoft Research. Microsoft SEAL (Simple Encrypted Arithmetic Library) v4.0. 2023 [cited 2025 Apr. 25]. Retrieved from: <https://github.com/microsoft/SEAL>
25. Microsoft. Open Enclave SDK. 2020 [cited 2025 Apr. 25]. Retrieved from: <https://openenclave.io>
26. Polyakov Y, Rohloff K, Schanck JM. PALISADE lattice cryptography library. 2017 [cited 2025 Apr. 25]. Retrieved from: <https://palisade-crypto.org>
27. Sabt M, Achemlal M. Trusted execution environment: what it is, and what it is not. 2015 IEEE Trustcom/BigDataSE/ISPA, 2015, 57-64. Doi: <https://doi.org/10.1109/Trustcom.2015.357>
28. Tsai C, Porter DE, Vij M. Graphene-SGX: A practical library OS for unmodified applications on SGX. USENIX Annu Tech Conf (USENIX ATC 17), 2017, 645-658. <https://www.usenix.org/conference/atc17/technical-sessions/presentation/tsai>
29. Weinberg Z, Felt AP, Hanna S, Song D. Measuring and mitigating privacy risks in third-party advertising libraries. In: Proceedings of the 10th ACM Workshop on Privacy in the Electronic Society, 2011, 207-214. Doi: <https://doi.org/10.1145/2046556.2046581>
30. Zhang Y, Juels A, Reiter MK, Ristenpart T. Cross-VM side channels and their use to extract private keys. In:

Proceedings of the 2012 ACM Conference on Computer and Communications Security, 2012, 305-316. Doi: <https://doi.org/10.1145/2382196.2382230>

31. Zhou S, Wang H, Liu Y, Li H. A survey of homomorphic encryption for machine learning. IEEE Trans Inf Forensics Secur. 2021; 16:4113-4126. Doi: <https://doi.org/10.1109/TIFS.2021.3115690>